

Algorithm for file updates in Python

Project description

A healthcare company uses an allow list of IP addresses that are allowed to access restricted information. The company also has a remove list containing IP addresses that are no longer allowed to access restricted information. The algorithm reads the allow list, identifies the IP addresses that appear in the remove list, removes them, and updates the original file.

Open the file that contains the allow list

The first step is to assign the name of the file to the `import_file` variable and use the `with` statement with the `open()` function and the `"r"` parameter to open the file for reading.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

Read the file contents

The `.read()` method is used to read the imported file and store its contents in the `ip_addresses` variable. The contents are then displayed to examine the data in its current format.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()
# Display `ip_addresses`
print(ip_addresses)
```

Convert the string into a list

After reading the file, the `ip_addresses` variable contains the IP addresses as a string. The `.split()` method is used to convert the string into a list so that each IP address can be processed individually.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Display `ip_addresses`
print(ip_addresses)
```

Iterate through the remove list

A `for` loop is used to iterate through the `ip_addresses` list. The loop variable is named `element`, and each element is displayed during every iteration.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:
    # Display `element` in every iteration
    print(element)
```

Remove IP addresses that are on the remove list

A conditional statement is placed inside the `for` loop. It checks whether the current `element` is in the `remove_list`. If it is, the `.remove()` method removes that element from `ip_addresses`.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Display `ip_addresses`
print(ip_addresses)
```

The `in` operator checks whether the current IP address is included in the `remove_list`. The `.remove()` method then removes it from the `ip_addresses` list.

Update the file with the revised list of IP addresses

The updated `ip_addresses` list must be converted back into a string before it can be written to the file. The `.join()` method is used with a space as the separator. The original file is then opened in `"w"` mode and rewritten with the updated list.

```

# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)

```

Summary

In this activity, I developed an algorithm to update an IP allow list using Python. I opened and read the `allow_list.txt` file and converted its contents from a string into a list. I then used a `for` loop and conditional statement to identify IP addresses that were included in the `remove_list` and removed them from the allow list. Finally, I converted the updated list back into a string and rewrote the original file with the updated information.